

Optimal Control in Systems with Feedback Loop

Веприков Андрей Сергеевич
МФТИ, Москва
veprikov.as@phystech.edu

Хританков Антон Сергеевич
МФТИ, Москва
prog.autom@gmail.com

Аннотация

В данной работе исследуется задача оптимального управления в системах машинного обучения (МО), с петлей обратной связи. Это означает, что состояние системы с течением времени становится зависимым от результатов работы алгоритма МО. Мы хотим управлять процессом обучения модели таким образом, чтобы в пределе получить заданное распределение состояний системы. Для математической постановки и решения данной задачи предлагается использовать уже ранее полученные результаты в области моделирования систем МО с петлей обратной связи с помощью динамических систем, а также аппарат теории стохастического оптимального управления.

Ключевые слова: Стохастическое управление, петли обратной связи.

1 Введение

Пусть у нас в системе возможен эффект петель обратной связи, то есть когда предсказания модели МО на шаге $t - \tau$ каким то образом влияют на обучаемую выборку на шаге t . В данной работе за время t будем обозначать условную меру времени, когда у нас меняется распределение данных в системе. Также мы предполагаем, что в системе можно вводить управление $u_t \in \mathcal{U}$, чтобы каким то образом влиять на модель МО (но не на данные напрямую). Идея работы состоит в том, что в условиях петли обратной связи можно косвенно влиять на распределение данных системы, воздействуя только на модель МО. Схематическая постановка задачи показана на Рисунке 1.

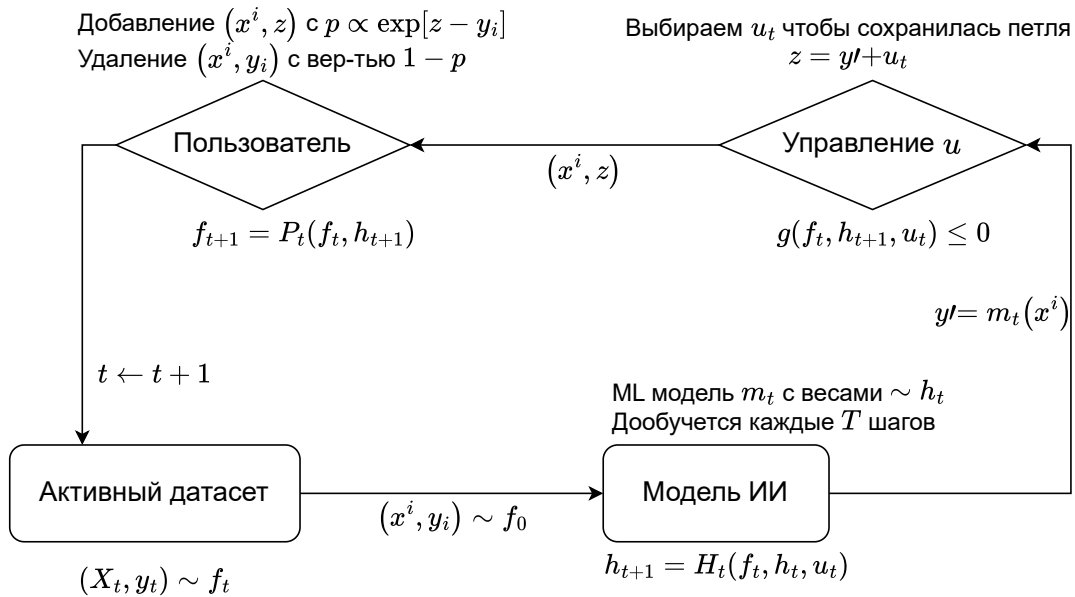


Рис. 1: Модельная задача оптимального управления в системе ИИ с петлей обратной связи. [TODO] Убрать цены и сделать более абстрактно.

1.1 Постановка задачи в терминах (Po)MDP

Определим пятерку: $(\mathcal{S}, \mathcal{U}, \mathbf{D}, \rho, \gamma)$, где

1. $\mathcal{S}$ – состояния
2. $\mathcal{U}$ – действия
3. $\mathbf{D}$ – оператор эволюции
4. r – награда
5. $0 < \gamma < 1$ – дисконтирующий коэффициент

Задача ставится как

$$\max_{u_1, \dots, u_\infty} \sum_{t=0}^{+\infty} \gamma^t r(s_t, u_t), \quad (1)$$

при условии

$$s_{t+1} = \mathbf{D}(s_t, u_t). \quad (2)$$

В нашей задаче вводится ряд уточнений по сравнению с постановкой MDP:

1. Ограничение на управление на каждом шаге
2. Оператор эволюции разбивается на 2 части: ответ пользователей и обучение агента

1.2 Математическая постановка задачи

1. $(f_t, h_t) \in \mathcal{S}$ – функции плотности состояний системы и распределения весов ML модели
2. $u_t \in \mathcal{U}$ – доступные управления
- 3(a). $P_t \in \mathbf{P}$ – оператор эволюции плотностей состояний системы f_t (решения пользователей)
- 3(b). $H_t \in \mathbf{H}$ – оператор эволюции плотности весов ML модели h_t (алгоритм обучения)
4. $r(f_t)$ – функция награды за текущее состояние системы
5. $0 < \gamma < 1$ – дисконтирующий коэффициент
6. $g(f_t, h_{t+1})$ – функция, показывающая, что в системе с данными $\sim f_t$ и весами модели $\sim h_{t+1}$ при нулевом дальнейшем управлении $u_{s \geq t} \equiv 0$ траектория $\{f_s, h_{s+1}\}_{s=t}^\infty$ нас удовлетворяет (что значит удовлетворяет будет определено позже)

Что нам не доступно	Что нам доступно
$f_t \in \mathcal{S}$ $P_t \in \mathbf{P}$	Ограниченная выборка из $f_t \in \mathcal{S}$ $H_t \in \mathbf{H}, u_t \in \mathcal{U}, g, \gamma$

1.3 Математическая постановка задачи

Оптимизационная задача:

$$\min_{u_1, \dots, u_\infty} \sum_{t=0}^{+\infty} \gamma^t r(f_t, h_{t+1}), \quad (3)$$

при условии

$$h_{t+1} = H_t(f_t, h_t, u_t), \quad H_t \in \mathbf{H}, \quad (4)$$

$$g(f_t, h_{t+1}) \leq 0, \quad (5)$$

$$f_{t+1} = P_t(f_t, h_{t+1}), \quad P_t \in \mathbf{P}, \quad (6)$$

Уравнение (5) может быть переписано в виде $\mathbb{P}\{g(\mathbf{x}, \boldsymbol{\theta}) \leq 0\} \geq 1 - \delta$, где $\mathbf{x} \sim f_t$ и $\boldsymbol{\theta} \sim h_{t+1}$, то есть:

$$\int_{\mathbf{x}} \int_{\boldsymbol{\theta}} \mathbf{1}\{g(\mathbf{x}, \boldsymbol{\theta}) \leq 0\} f_t(\mathbf{x}) h_{t+1}(\boldsymbol{\theta}) d\mathbf{x} d\boldsymbol{\theta} \geq 1 - \delta. \quad (7)$$

1.4 Жадный алгоритм управления через ω -предельное множество

1. Пусть мы находимся на шаге t . Нам доступны $\hat{f}_t$, h_t и $\mathbf{H}$.
2. Из всех возможных управлений $u_t \in \mathcal{U}$ выбираем те, которые сохраняют петлю обратной связи, то выполнено ограничение (5)
3. Для каждого подходящего управления находим, какие распределения весов $\{h_{u_t}^\infty\}$ могут быть у модели в пределе, если на шагах $t+1, \dots, \infty$ мы не будем управлять.
4. Для задачи обучения с учителем, так как в системе присутствует петля обратной связи, плотность данных $f_{u_t}^\infty$ для управления u_t в пределе будет иметь вид $f_{u_t}^\infty(\mathbf{x}, \mathbf{y}) > 0 \Leftrightarrow \mathbf{y} = h_{u_t}^\infty(\mathbf{x})$ для всех примеров состояния системы $(\mathbf{x}, \mathbf{y})$
5. Выбираем u_t , которое максимизирует $r(f_t)$
6. Переходим на шаг $t+1$

2 Пример для Схемы на Рис. 1

Рассмотрим условия (4), (6) и (5) на модельном эксперименте из статей [CITE], представленном на рисунке 1. Для каждого из уравнений (4), (6) и (5) будет предложена свой параграф.

2.1 Эволюция распределения весов модели (уравнение (4))

$$h_{t+1} = H_t(f_t, h_t, u_t), \quad H_t \in \mathbf{H}.$$

В примере мы будем обучать веса следующим образом:

$$h_{t+1} = \begin{cases} h_t & \text{если } t \text{ не делится на } T \\ H(f_t, u_t) & \text{иначе} \end{cases},$$

где T – сколько итераций переобучаемся и $H(f_t, u_t)$ – обучение модели $m_\theta(\cdot)$ с данными $\sim f_t$ и управлением u_t .

Как можно вводить это управление u_t ? Например можно рассмотреть его как коэффициент регуляризации к какому-то «хорошим» весам $\Theta_t^{\mathcal{F}}$. То есть h_{t+1} – это функция плотности весов θ^{t+1} модели, обученная по правилу:

$$\theta^{t+1} \in \arg \min_{\theta \in \Theta} \left\{ \text{Loss}[m_\theta(\cdot), f_t] + \frac{u_t}{2} \cdot r(\theta, \Theta_t^{\mathcal{F}}) \right\},$$

где $r(\theta, \Theta_t^{\mathcal{F}})$ – расстояние до «хороших» весов $\Theta_t^{\mathcal{F}}$, которые будут определены позже.

2.2 Эволюция данных системы (уравнение (6))

$$f_{t+1} = P_t(f_t, h_{t+1}), \quad P_t \in \mathbf{P}.$$

Как работает «среда»? С вероятностью $p_t := \text{usage}_t$ мы заменяем один элемент из выборки длины N на предсказание модели с весами $\theta \sim h_{t+1}$ с шумом, а с вероятностью $1 - p_t$ мы ничего не делаем. Формализовать это можно таким образом, пусть $\xi \sim \text{Be}(p_t)$, тогда по формуле полной вероятности имеем [1/N???

$$\begin{aligned} f_{t+1}(\cdot) &= \frac{1}{N} [\mathbb{P}[\xi = 0] \cdot f_{t+1}(\cdot | \xi = 0) + \mathbb{P}[\xi = 1] \cdot f_{t+1}(\cdot | \xi = 1)] + \left(1 - \frac{1}{N}\right) f_t(\cdot) \\ &= \left(\frac{1 - p_t}{N} + \frac{N - 1}{N}\right) \cdot f_t(\cdot) + \frac{p_t}{N} \cdot G_t(h_{t+1}, f_t)(\cdot), \end{aligned} \quad (8)$$

$$= \left(1 - \frac{p_t}{N}\right) \cdot f_t(\cdot) + \frac{p_t}{N} \cdot G_t(h_{t+1}, f_t)(\cdot), \quad (9)$$

где $G_t(h_{t+1}, f_t)$ – оператор, который переводит плотность весов модели h_{t+1} и данных f_t , доступных системе в плотность данных. В эксперименте на Рис. 1 мы добавляем шум к предсказаниям модели, то есть:

$$G_t(h_{t+1}, f_t) = \text{свертка} \left[\begin{pmatrix} m_{\theta^{t+1}}(\mathbf{x}) \\ \mathbf{x} \end{pmatrix} ; \begin{pmatrix} \mathcal{N}(0, \text{adherence}_t^2) \\ \text{шум на } \mathbf{x} \end{pmatrix} \right], \quad \text{где } \mathbf{x} \sim f_t \text{ и } \theta^{t+1} \sim h_{t+1}. \quad (10)$$

2.3 Условие на сохранение петли обратной связи (уравнение (7))

$$g(f_t, h_{t+1}) \leq 0.$$

Объясним что такое распределение данных f_∞ , которое нас удовлетворяет. Самое базовое наше требование – это сохранение петли обратной связи. То есть в функцию g всегда "защита" эта петля. Однако g может включать в себя и другие требования, такие как не вырождаемость системы (смотри Параграф 2.5). Мы вводим предположение:

Предположение 1. Для начальных распределений данных системы f_0 и весов модели МО выполнено, что $g(f_0, h_0) \leq 0$.

Это предположение аналогично тому, что в constrained итеративной оптимизации начальная точка берется из допустимого множества. Предположение 1 говорит, что на каждом шаге t множество доступных управлений не пусто, то так как всегда можно выбрать $u_t = 0$ (доказательство по индукции).

2.4 Итоговый Алгоритм

Предположения:

1. Если $g_t(f_t, h_{t+1}) \leq 0$, тогда $\text{usage}_t = \text{const}$, то есть usage_t зависит от h_{t+1} и f_t (от метрик) как функция барьера. Причем если $u_t = 0$, тогда $g_t(f_t, h_{t+1}) \leq 0$.
2. $\text{adherence}_t = \text{const}$, то есть она не зависит от h_{t+1} и f_t .

Если выполнено Предположение 2, тогда в уравнении (10) на оператор плотности $G_t(h_{t+1}, f_t)$ управление влияет только на $m_{\theta_{t+1}}(\cdot)$, соответственно мы можем менять только распределение целевой переменной y в системе. Назовем обрезанное только по целевой переменной y желаемое множество функций распределения $\mathcal{F}$ как $\mathcal{F}|_y$.

Так как в уравнении (9) на обновление f_{t+1} вероятность $p_t = 0$ или const , мы управляем только близостью $G_t(h_{t+1}, f_t)$, нам нужно приближать $m_{\theta_{t+1}}(\cdot)$ к $\mathcal{F}|_y$. За это и отвечает множество «хороших» $\Theta_t^{\mathcal{F}}$, то есть $\Theta_t^{\mathcal{F}}$ – это идеально возможные веса θ , чтобы данные вида $(m_\theta(\mathbf{x}^t), \mathbf{x})^T$, $\mathbf{x} \sim f_t$ попадали в $\mathcal{F}$.

Для данного примера системы из Рис.1 и при выполнении предположений выше, жадный алгоритм управления из Раздела 2.4 будет выглядеть следующим образом:

1. На шаге t у нас имеется выборка (датасет) $\{(x^i, y_i)\}_{i=1}^n$
2. Находим $\Theta_t^{\mathcal{F}}$:

$$\Theta_t^{\mathcal{F}} = \arg \max_{\theta \in \Theta} \left\{ \sum_{i=1}^n \mathbf{1} [m_\theta(x^i) \in \mathcal{F}|_y] \right\}.$$

Идеальный вариант, если $\Theta_t^{\mathcal{F}}$ не зависит от t , то есть мы заранее (на 0-ой итерации) знаем желаемые веса модели, или какие веса дают желаемый таргет.

3. Выбрать достаточно маленькое $u_t \in \mathbb{R}_+$.
4. Обучить модель с помощью функции ошибки вида

$$\theta^{t+1} \in \arg \min_{\theta \in \Theta} \left\{ \text{Loss}[m_\theta(\cdot), f_t] + \frac{u_t}{2} \cdot r(\theta, \Theta_t^{\mathcal{F}}) \right\}$$

5. Если метрики модели $m_{\theta^{t+1}}$ меньше, чем $1 - \delta_t$, тогда положить $u_t = u_t/2$ и вернуться на предыдущий шаг. Иначе переходить на шаг $t = t + 1$.

2.5 Деградация Системы

Важным эффектом, который не был рассмотрен пока что в этой работе это так называемая «деградация системы». Допустим мы постоянно повышаем цены на товары, соответственно каждый шаг число покупателей уменьшается, даже если им нравятся наши рекомендации. Таким образом, если мы продолжим увеличивать цены, то на шаге $t \gg 1$ у нас просто не будет новых покупателей и сделок, хотя на всех шагах до этого метрики модели были большими.

Формализовать этот эффект можно следующим образом. Следуя идеям из Po-MDP [CITE] можно формализовать смерть системы через «уверенность» в том, что мы находимся в каком то состоянии. В нашем случае это будет просто $\|\hat{f}_t - f_t\|^2$, то есть чем лучше мы можем оценить распределение данных системы на шаге t , тем больше активных пользователей у нас есть. Это же можно эквивалентно формализовать таким (более простым и прикладным) способом: пусть N_t – это выборка (данные системы), полученные именно на шаге t , тогда если $N_t \geq N_{t+1}$ или же $N_t/N_{t-1} \geq 1 + \delta$, тогда система не приближается к смерти, если же $N_t \rightarrow 0$, тогда система стремится к смерти и нам нужно менять управление.

Результаты эксперимента на Рис. 2 показывают, что при жадном управлении, описанным в Секции 2.4 наблюдается процесс деградации системы, хотя метрики качества обученной модели МО растут (красный алгоритм). Проблему можно решить, модифицировав функцию g из постановки задачи (5), введя в нее зависимость от f_t – оценки плотности данных на шаге t .

Таким образом, можно получить более аккуратный алгоритм, который учитывает текущую уверенность в положении агента в системе. Если отличие между f_t и $\hat{f}_t$ незначительно, система остается стабильной, и возможно применение жадного управления. В противном случае требуется корректировка стратегии, чтобы избежать деградации. В эксперименте это отличие можно аппроксимировать через количество остающихся элементов в выборке N_t , где «выживаемость» пользователей из исходного набора может быть представлена как зависимость от размера выборки. Чем больше N_t , тем меньше влияет управление в одной сделке на всех остающихся пользователей. Как показано на Рис. 2, такой подход позволяет избежать деградации системы, при этом приближая распределение данных к целевому.

Вычислительный эксперимент. Для демонстрации данного эффекта в работе проводится вычислительный эксперимент. На каждом шаге t в системе представлен активный датасет размером N_t , содержащий признаки объектов недвижимости X_t и их цены y_t . Процесс обновления данных происходит следующим образом: случайно выбирается индекс $i \sim U(1, N_0)$, вычисляется предсказание модели: $z = m_t(X_0[i]) + u_t$, где m_t — модель МО, дообучаемая каждые T шагов и u_t — управление. Целевая переменная $y_0[i]$ заменяется на z , и элемент $(X_0[i], z)$ добавляется в активный датасет с вероятностью $p \propto \exp(-(z - y_0[i]))$, а с вероятностью $1 - p$ элемент i удаляется из (X_t, y_t) .

Данный механизм моделирует поведение пользователей: с вероятностью p они следуют предсказаниям модели, но при высокой цене вероятность покупки снижается. Целью управления является повысить цены как можно больше.

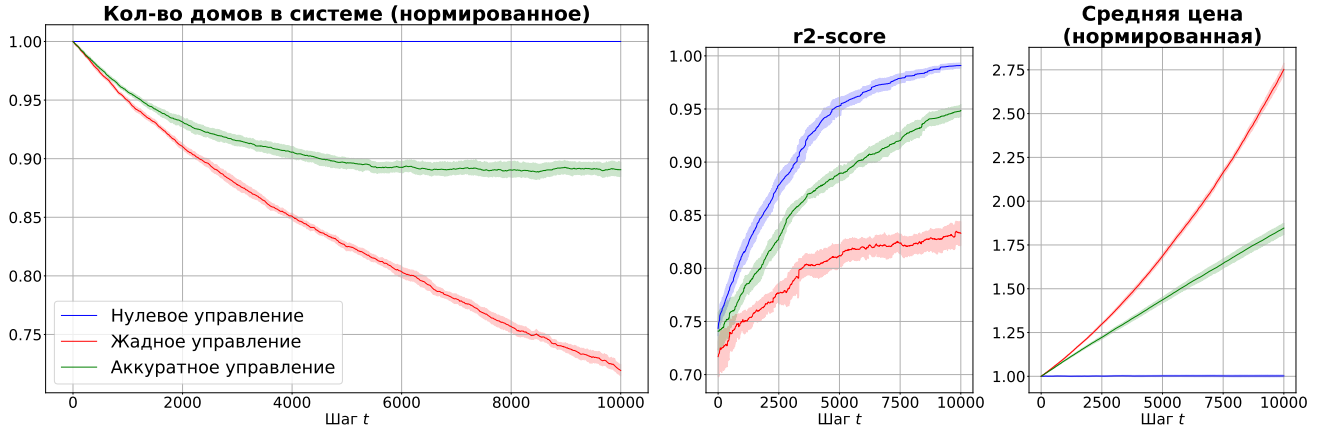


Рис. 2: Моделируемый эксперимент, в котором видно, что жадное управление ведет к деградации системы.

3 Generative Replay

В multi task постановке зачастую применяются методы под названием Generative Replay, основная идея которого состоит в том, что модели на шаге i в обучающую выборку «подмешивается» результаты ее работы с предыдущих шагов, то есть ее функция потерь выглядит как

$$L_{train}(\theta_{t+1}) = u \underbrace{\mathbb{E}_{(\mathbf{x}, \mathbf{y}) \sim \mathcal{D}_t} [L(S(\mathbf{x}; \theta_{t+1}), \mathbf{y})]}_{\text{new data}} + (1 - u) \underbrace{\mathbb{E}_{\mathbf{x}' \sim G_{t-1}} [L(S(\mathbf{x}'; \theta_{t+1}), S(\mathbf{x}'; \theta_t))]}_{\text{replay data}}.$$

Постановку generative replay мы можем переписать в виде

$$L_{train}(\theta_{t+1}) = \mathbb{E}_{(\mathbf{x}, \mathbf{y}) \sim \mathcal{D}_t} [L(S(\mathbf{x}; \theta_{t+1}), \mathbf{y})],$$

где $\mathcal{D}_t$ — объединение датасетов $\mathcal{D}_t$ и генератора G_{t-1} с соотношением u .

Мы будем моделировать обучение через generative replay через постановку sliding window. На 1-ом шаге $\mathcal{D}_1 = \mathcal{D}_1$. Пусть у нас есть n заданий и суммарная длина датасетов равна $T = \sum_{i=1}^n |D_i|$. Далее каждый шаг $t \in [1; T]$ происходит следующее:

- Удаляем первый элемент $\mathcal{D}_t[0]$ (самый старый)
- С вероятностью u_t : $(\mathbf{x}, \mathbf{y}) \sim \mathcal{D}_{t//n}$ (берем элемент из нового датасета)
- С вероятностью $1 - u_t$: $\mathbf{x} \sim G_{t//n}$ и $\mathbf{y} = S_{t//n}(\mathbf{x})$ (берем элемент из предсказания и таргет как выход предыдущей модели S)
- В конец текущего окна $\mathcal{D}_t$ кладем $(\mathbf{x}, \mathbf{y})$.
- Если $t \% n == 0$, дообучаем (S_t, G_t) на окне $\mathcal{D}_{t//n}$.

Используемые расписания:

- $u_t = \frac{1}{(t//n)+1}$. В этом случае для каждого датасета r на шаге обучения будет одинаковое кол-во данных от датасетов 1.. r . Выполняется $u_0 = 1, u_T = 1/T \neq 1$.

- $u_t = u_0 = \text{const}$
- $u_t = \exp\left[\frac{t}{T}\right]$
- $u_t = 1 - u_0 * t^\alpha, u_t = 1 - \frac{u_0}{t^\alpha + 1}$.

Существующие гипотезы:

- Используя меньший объем одновременно используемых (доступных) данных можно добиться такого же качества, как и обучение на всех данных сразу
- Расписание u_t влияет на обучение
- Влияет также структура модели
- Шум в добавляемых данных также влияет на u_t

3.1 Эксперименты

Парабола. В первом эксперименте мы разбили параболу на n частей и сформировали n разных датасетов:

$$D_r = \{(x_i^r, y_i^r = (x_i^r)^2)\}_{i=1}^{T//n} ; x_i^r \sim \text{Uniform}[r; r+1] \text{ для всех } r \in \overline{1, n}.$$

В качестве модели используем линейную регрессию, которую обучаем с SGD. Таким образом, в этом эксперименте $L_{train} = 0$ не достигается (тк данные квадратичные).

Результаты:

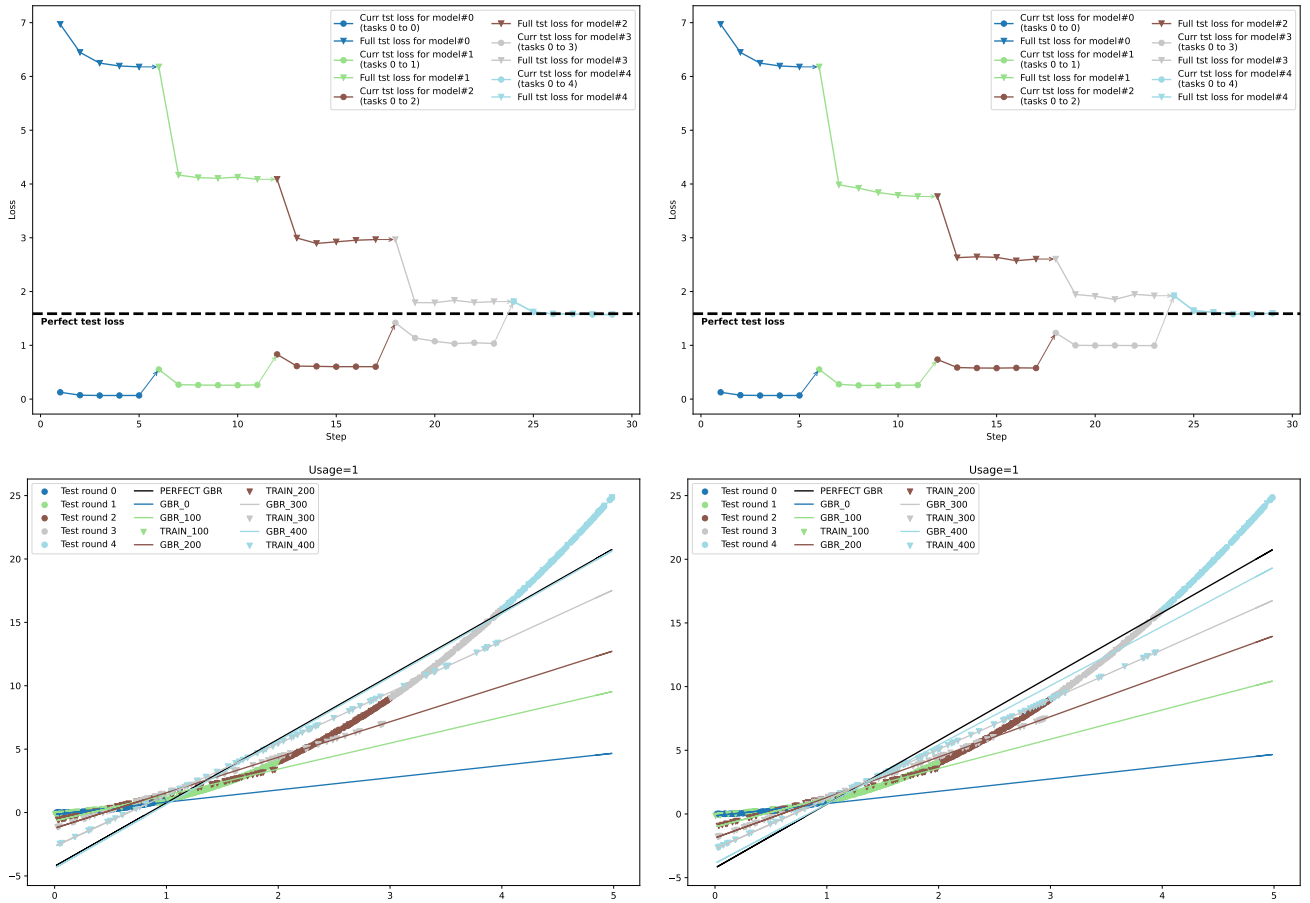


Рис. 3: Результаты эксперимента с параболой. Слева направо: inv , exp

SVD. Генерируем матрицу $X \in \mathbb{R}^{(n \cdot N) \times n}$, берем ее SVD разложение (предполагаем, что $N > 1$ и $\text{rank}(X) = n$): $X = \sum_{r=1}^n \sigma_r u_r v_r^T$. Тогда датасет $D_r = (X = \sigma_r u_r v_r^T, Y = \sigma_r u_r)$. Таким образом, чтобы достичь нулевой ошибки для задачи r можно выучить любой вектор вида $w = v_r + \sum_{r'=1, r' \neq r}^n \alpha_{r'} v_{r'}$ для любых $\alpha_{r'}$. Для того чтобы получить нулевой лосс на всех задачах, нужно выучить вектор $w = \sum_{r=1}^n v_r$.

Результаты:

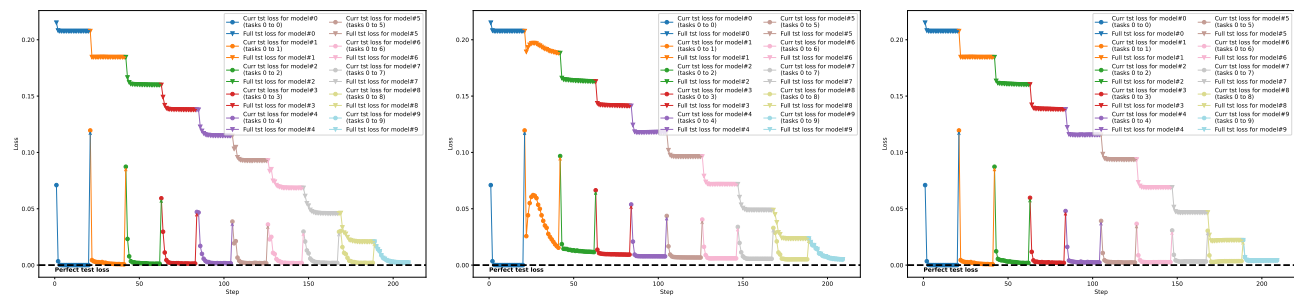


Рис. 4: Результаты SVD эксперимента. Слева направо: inv, exp, const

Приложение